Kerman, Sara J. (Fed)

From: Lorenz Panny <l.s.panny@tue.nl>
Sent: Thursday, December 21, 2017 5:00 PM
To: pqc-comments
Cc: pqc-forum@list.nist.gov
Subject: OFFICIAL COMMENT: Guess Again

Follow Up Flag: Follow up
Flag Status: Completed

Dear all,

the following Python script quickly recovers the message from a given "Guess Again" ciphertext without knowledge of the private key:

https://yx7.cc/files/guessedonce.py.txt

I have only tried the attack on the ciphertexts in the known-answer tests file so far, but I think there is no reason to believe that it does not work in general.
Notice that the attack is solely based on statistical properties of the ciphertext and does not even require the public key. The script expects the contents of the KAT archive in the same direc- tory (but can easily be modified to decrypt other ciphertexts).

-- Lorenz

Attack script — guessedonce.py (Lorenz Panny)

Python attack script by Lorenz Panny, referenced by the OFFICIAL COMMENT on the preceding page (21 December 2017). It recovers the plaintext from a "Guess Again" ciphertext using only statistical properties of the ciphertext (a per-bit mean-comparison of the record start-points grouped by label) — requiring neither the public nor the private key. Source: https://yx7.cc/files/guessedonce.py.txt

```
#!/usr/bin/env python3
import binascii, struct

def recover_bit(ct, bit):
    assert bit < len(ct) // 4000
    ts = [struct.unpack('BB', ct[i:i+2]) for i in range(4000*bit, 4000*(bit+1), 2)]
    xs, ys = [a for a, b in ts if b == 1], [a for a, b in ts if b == 2]
    return sum(xs) / len(xs) >= sum(ys) / len(ys)

def decrypt(ct):
    res = sum(recover_bit(ct, b) << b for b in range(len(ct) // 4000))
    return int.to_bytes(res, len(ct) // 4000 // 8, 'little')

kat = 0
for l in open('KAT_GuessAgain/GuessAgainEncryptKAT_2000.rsp'):

    if l.startswith('msg = '):

        # only used for verifying the recovered plaintext.
        msg = binascii.unhexlify(l[len('msg = '):].strip())

    elif l.startswith('c = '):

        ct = binascii.unhexlify(l[len('c = '):].strip())

        print('{}attacking known-answer test #{}'.format('\n' * (kat > 0), kat))
        print('correct plaintext:   {}'.format(binascii.hexlify(msg).decode()))

        plain = decrypt(ct)

        print('recovered plaintext: {} ({})'.format(binascii.hexlify(plain).decode(), plain ==
    msg))

        kat += 1
```